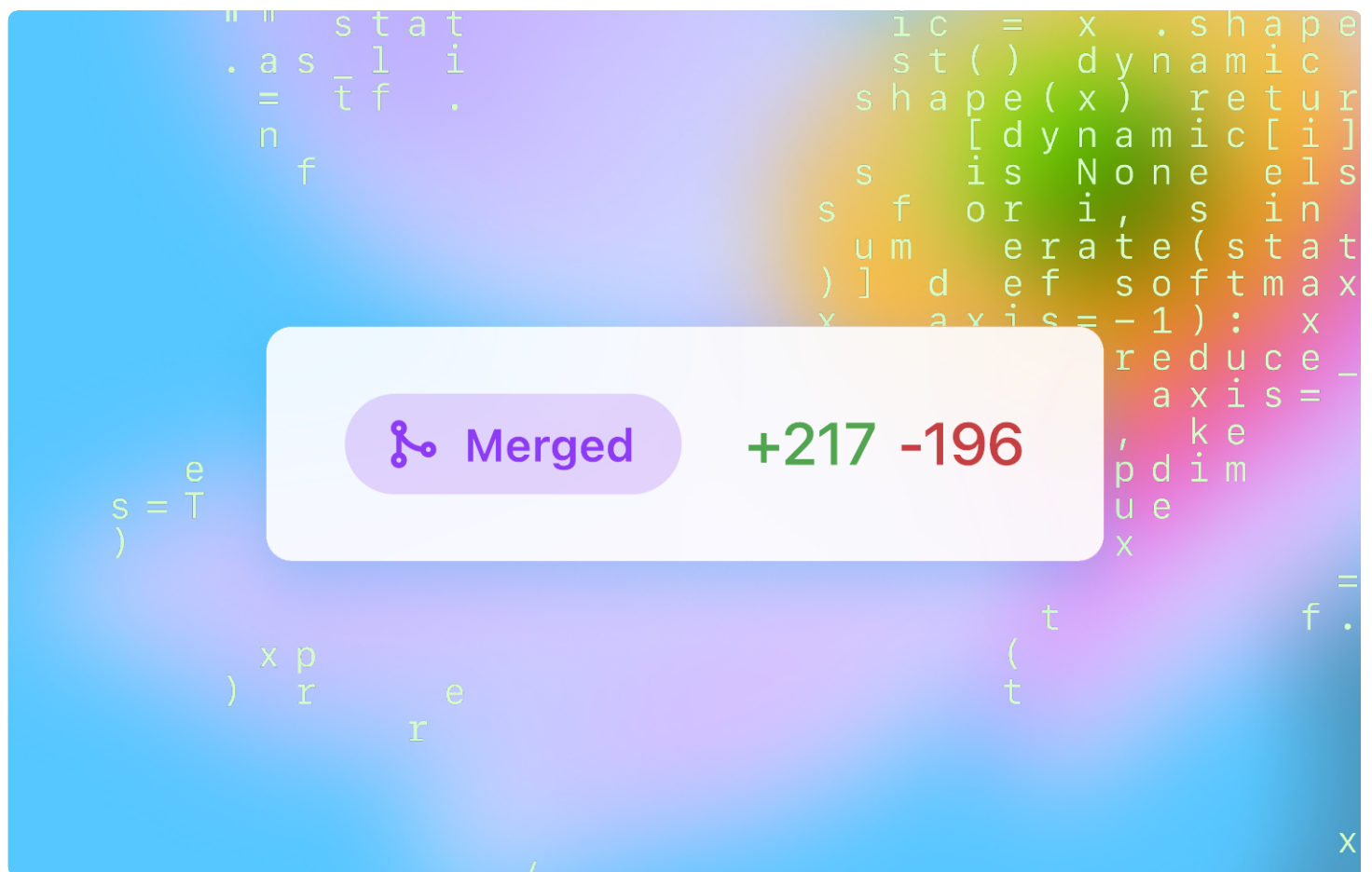


OpenAI

Как OpenAI использует Codex



Содержание

Введение	3
Примеры использования	
Понимание кода	4
Рефакторинг и миграции	5
Оптимизация производительности	6
Улучшение тестового покрытия	7
Ускорение разработки	8
Сохранение рабочего потока	9
Исследование и генерация идей	10
Лучшие практики	11
Что дальше	12

Введение

Codex ежедневно используют многие технические команды OpenAI, включая Security, Product Engineering, Frontend, API, Infrastructure и Performance Engineering. Команды применяют его, чтобы ускорять широкий спектр инженерных задач: от понимания сложных систем и рефакторинга больших кодовых баз до выпуска новых функций и устранения инцидентов в условиях жёстких сроков.

На основе интервью с инженерами OpenAI и внутренних данных об использовании мы собрали сценарии и лучшие практики, которые показывают, как Codex помогает нашим командам двигаться быстрее, повышать качество работы и управлять сложностью на масштабе.

Понимание кода

Codex помогает нашим командам быстро разобраться в незнакомых участках кодовой базы при онбординге, отладке или расследовании инцидента.

Команды часто используют Codex, чтобы найти основную логику функции, проследить связи между сервисами или модулями и пройти путь данных через систему. Он также помогает выявлять архитектурные паттерны и недостающие фрагменты документации, на подготовку которых вручную иначе потребовалось бы много усилий.

Во время реагирования на инциденты Codex помогает инженерам быстро входить в новые области: показывает взаимодействия между компонентами или отслеживает, как состояния отказа распространяются по системам.

Истории от наших команд

Когда исправляю баг, я использую Ask mode, чтобы понять, где ещё в кодовой базе может проявиться та же проблема.

**Инженер по
производительности,
Retrieval Systems**

Когда я на дежурстве, вставляю stack trace и прошу Codex показать, где живёт auth flow. Он сразу выводит на нужные файлы, так что triage проходит быстро.

**SRE,
API Platform**

Codex намного быстрее grep отвечает на мои вопросы по репозиторию в стиле «где это сделать?» - и по Terraform, и по Python.

**DevOps-инженер,
Infrastructure Services**

Попробуйте использовать Codex для понимания кода с такими промптами:

- ☐ Где в этом репозитории реализована логика аутентификации?
- ☐ Кратко опиши, как запрос проходит через этот сервис: от точки входа до ответа.
- ☐ Какие модули взаимодействуют с [вставьте имя модуля] и как обрабатываются отказы?

Рефакторинг и миграции

Codex часто используют для изменений, которые затрагивают несколько файлов или пакетов. Например, когда инженеры обновляют API, меняют способ реализации паттерна или переходят на новую зависимость, Codex помогает применять изменения последовательно.

Он особенно полезен, когда одно и то же обновление нужно внести в десятки файлов или когда изменение требует понимания структуры и зависимостей, которое сложно поймать регулярным выражением или поиском с заменой.

Его также используют для чистки кода: разбиения слишком больших модулей, замены устаревших паттернов современными и подготовки кода к лучшей тестируемости.

Истории от наших команд

Codex заменил каждый legacy `getUserByld()` на новый сервисный паттерн и открыл PR. То, что заняло бы часы, он сделал за минуты.

**Backend Engineer,
ChatGPT Web**

Чтобы снять блокиеры перед запуском, я прошу Codex найти все экземпляры старого паттерна, кратко описать влияние в Markdown, а затем открыть PR с исправлениями.

**Product Engineer,
ChatGPT Enterprise**

Попробуйте использовать Codex для рефакторинга и миграций с такими промптами:

- Разбей этот файл на отдельные модули по зонам ответственности и сгенерируй тесты для каждого.
- Переведи весь callback-based доступ к базе данных на `async/await`.

Оптимизация производительности

Codex используют для поиска и устранения узких мест производительности.

Во время настройки или повышения надёжности инженеры просят Codex проанализировать медленные или требовательные к памяти пути выполнения: неэффективные циклы, лишние операции или дорогие запросы - и предложить оптимизированные альтернативы. Это часто даёт заметный выигрыш в эффективности и надёжности.

Codex также помогает поддерживать здоровье кода: выявляет рискованные или устаревшие паттерны, которые всё ещё активно используются. Команды опираются на него, чтобы снижать долгосрочный технический долг и заранее предотвращать регрессии.

Истории от наших команд

*Я использую Codex, чтобы искать повторяющиеся дорогие вызовы к БД. Он хорошо отмечает *hot paths* и набрасывает *batched queries*, которые я потом довожу.*

**Infrastructure Engineer,
API Reliability**

Codex отлично быстро замечает проблемы производительности - я экономлю 30 минут работы, потратив 5 минут на промпт.

**Platform Engineer,
Model Serving**

Попробуйте использовать Codex для оптимизации производительности с такими промтами:

- ☐ Оптимизируй этот цикл по памяти и объясни, почему твоя версия быстрее.
- ☐ Найди повторяющиеся дорогие операции в этом request handler и предложи возможности для кэширования.
- ☐ Предложи более быстрый способ пакетировать DB-запросы в этой функции.

Улучшение тестового покрытия

Codex помогает инженерам быстрее писать тесты - особенно там, где покрытие слабое или полностью отсутствует.

Когда инженеры работают над исправлением бага или рефакторингом, они часто просят Codex предложить тесты, покрывающие граничные случаи или вероятные пути отказа. Для нового кода он может генерировать unit- или integration-тесты на основе сигнатуры функции и окружающей логики.

Codex особенно полезен для выявления граничных условий: пустых входных данных, максимальной длины или необычных, но допустимых состояний, которые часто пропускают в первых тестах.

Истории от наших команд

Я направляю Codex на модули с низким покрытием на ночь и утром просыпаюсь с PR, где уже есть запускаемые unit-тесты.

**Frontend Engineer,
ChatGPT Desktop**

Когда переключение веток в монорепозитории болезненно, я прошу Codex написать тесты и запустить CI, пока продолжаю работать в своей ветке.

**Backend Engineer,
Payments & Billing**

Попробуйте использовать Codex для улучшения тестового покрытия с такими промптами:

- ☐ Напиши unit-тесты для этой функции, включая edge cases и пути отказа.
- ☐ Сгенерируй property-based test для этой утилиты сортировки.
- ☐ Расширь этот тестовый файл, чтобы покрыть пропущенные сценарии вокруг null-входов и невалидных состояний.

Ускорение разработки

Codex помогает командам двигаться быстрее, ускоряя как начало, так и завершение цикла разработки.

Когда начинается работа над новой функцией, инженеры используют его для каркаса boilerplate-кода: генерации папок, модулей и API-заглушек, чтобы быстро получить запускаемый код без ручного соединения каждой детали.

Когда проект приближается к релизу, Codex помогает укладываться в жёсткие сроки, беря на себя небольшие, но важные задачи: triage багов, закрытие последних пробелов в реализации, генерацию rollout-скриптов, telemetry hooks или конфигов.

Его также используют, чтобы превращать продуктовый фидбек в стартовый код. Инженеры часто вставляют запрос пользователя или спецификацию и просят Codex сгенерировать черновик, к которому позже можно вернуться и доработать.

Истории от наших команд

Я весь день был на встречах и всё равно смёржил 4 PR, потому что Codex работал в фоне.

**Product Engineer,
ChatGPT Enterprise**

Codex помог идеально доставить 3-4 низкоприоритетных фикса, которые иначе застряли бы в бэклоге, - это было очень вдохновляюще.

**Full-Stack Engineer,
Internal Tools**

Попробуйте использовать Codex для ускорения разработки с такими промптами:

- Создай каркас нового API route для POST /events с базовой валидацией и логированием.
- Сгенерируй telemetry hook для отслеживания успеха/ошибки нового onboarding flow, используя этот шаблон [вставьте пример вашего telemetry-кода].
- Создай stub-реализацию на основе этой спецификации: [вставьте спецификацию или продуктовый фидбек].

Сохранение рабочего потока

Codex помогает инженерам сохранять продуктивность, когда расписание разорвано и заполнено прерываниями.

Его используют, чтобы фиксировать незавершённую работу, превращать заметки в рабочие прототипы или выносить исследовательские задачи, к которым можно вернуться позже. Так проще ставить работу на паузу и продолжать без потери контекста, особенно во время дежурств или при большом количестве встреч.

Истории от наших команд

Если я замечаю маленький попутный фикс, я запускаю задачу для Codex вместо переключения веток и смотрю его PR, когда освобождаюсь.

**Backend Engineer,
ChatGPT API**

Я регулярно пересылаю Codex трейды из Slack, трейсы Datadog, issues и многое другое, чтобы оставаться сфокусированным на приоритетной работе.

**API Engineer,
Infrastructure Observability**

Попробуйте использовать Codex для сохранения рабочего потока с такими промптами:

- ☐ Составь план рефакторинга этого сервиса и разбиения его на меньшие модули.
- ☐ Набросай retry logic и добавь TODO - backoff logic я заполню позже.
- ☐ Кратко опиши этот файл, чтобы завтра я мог продолжить с того места, где остановился.

Исследование и генерация идей

Codex полезен и для открытых задач: поиска альтернативных решений или проверки архитектурных решений. Можно попросить его предложить разные способы решить проблему, исследовать незнакомые паттерны или стресс-тестировать предположения. Это помогает увидеть компромиссы, расширить набор вариантов дизайна и уточнить реализацию.

Его также используют для поиска связанных багов. Зная конкретную проблему или устаревший метод, Codex может найти похожие паттерны в других местах кода, что упрощает поиск регрессий и завершение чистки.

Истории от наших команд

Codex помогает мне решить проблему «холодного старта»: я вставляю spec и docs, а он создаёт каркас кода или показывает, что я забыл.

**Product Engineer,
ChatGPT Desktop**

После фикса бага я спрашиваю Codex, где могут скрываться похожие баги, а затем запускаю follow-up задачи.

**Performance Engineer,
Retrieval Systems**

Попробуйте использовать Codex для исследования и генерации идей с такими промптами:

- ☐ Как это работало бы, если бы система была event-driven, а не request/response?
- ☐ Найди все модули, которые вручную собирают SQL-строки вместо использования нашего query builder.
- ☐ Перепиши это в более функциональном стиле, избегая мутаций и побочных эффектов.

Лучшие практики

Codex работает лучше всего, когда ему дают структуру, контекст и пространство для итераций. Вот привычки, которые команды OpenAI развивают, чтобы стабильно получать от него пользу в ежедневной работе.

Начинайте с Ask Mode

При крупных изменениях сначала попросите Codex составить план реализации в Ask mode. Затем этот план становится входными данными для следующих промптов, когда вы переключаетесь в Code Mode. Такой двухшаговый процесс удерживает Codex в контексте и помогает избегать ошибок в результате. Codex лучше всего работает с хорошо ограниченными задачами, которые заняли бы у вас или коллеги около часа или несколько сотен строк кода. По мере улучшения моделей размер задач, которые он может брать на себя, будет расти.

Итеративно улучшайте среду разработки Codex

Настройка startup script, переменных среды и доступа в интернет заметно снижает частоту ошибок Codex. По мере запуска задач обращайте внимание на ошибки сборки, которые можно исправить в конфигурации окружения Codex. На это может уйти несколько итераций, но в долгосрочной перспективе это даёт существенный выигрыш в эффективности.

Структурируйте промпт так, будто пишете GitHub Issue

Codex лучше отвечает, когда промпты похожи на то, как вы описываете изменение в PR или issue. Это значит, что стоит добавлять пути к файлам, названия компонентов, diff и фрагменты документации, когда они релевантны. Промпты по шаблону «Реализуй это так же, как сделано в [module X]» улучшают результат.

Используйте очередь задач Codex как лёгкий backlog

Запускайте задачи, чтобы фиксировать побочные идеи, частичную работу или случайные фиксы. Нет давления сгенерировать полноценный PR за один подход. Codex хорошо работает как промежуточная зона, к которой можно вернуться, когда вы снова в фокусе.

Используйте AGENTS.md для постоянного контекста

Поддерживайте файл AGENTS.md, чтобы Codex эффективнее работал в вашем репозитории от промпта к промпту. Обычно такие файлы включают правила именования, бизнес-логику, известные особенности или зависимости, которые Codex не может вывести только из кода. Подробнее о структуре AGENTS.md - в документации.

Используйте «Best of N», чтобы улучшать результат

Функция Best-of-N позволяет одновременно генерировать несколько вариантов ответа на одну задачу, быстро исследовать несколько решений и выбрать лучшее. Для более сложных задач можно просмотреть несколько итераций и объединить части разных ответов, чтобы получить более сильный результат.

Что дальше

Codex всё ещё находится в research preview, но уже заметно влияет на то, как мы разрабатываем: помогает двигаться быстрее, писать более качественный код и браться за работу, которая иначе никогда не получила бы приоритет.

Нас вдохновляет будущий потенциал: по мере того как модели становятся лучше, а Codex глубже интегрируется в наши рабочие процессы, мы рассчитываем открыть ещё более мощные способы разработки ПО с его помощью. Мы продолжим делиться тем, чему научимся на этом пути.

OpenAI

